

技术方案

0 编制说明

大模型训练中的 SSD 适配不能简单理解为“把数据写入 SSD”。真实训练系统中，参数、梯度、优化器状态、激活值、Checkpoint、数据集缓存和日志/Profile 数据具有不同的生命周期和访问模式。若训练框架不区分这些对象，而是以普通文件或无语义块 I/O 的方式混合写入 SSD，将导致训练侧难以控制小随机写、临时写、重复写和 Checkpoint 突发写对训练吞吐的影响，SSD 侧也难以区分短生命周期数据、长期训练状态和持久化恢复数据。

本方案的核心思路是将训练数据由“无差别 I/O 请求”提升为“带训练语义的数据流”。训练侧负责识别数据对象、输出语义标签、组织 I/O 形态、控制调度优先级，并提供可复现实验指标；AI SSD 侧可基于这些信息开展放置、调度、缓存和寿命相关优化。

1 发展现状

1.1 大模型训练对存储系统的新需求

1.1.1 显存容量瓶颈与分层存储趋势

大模型训练已经从单纯计算密集型任务演进为计算、显存、内存、存储和互连共同受限的复杂系统任务。训练过程中，GPU 显存不仅需要容纳模型参数，还需要容纳前向传播产生的激活值、反向传播产生的梯度、优化器状态、混合精度训练中的 FP32 master weight、通信缓冲区以及框架运行时产生的临时张量，使训练状态规模显著大于模型参数本身。

表1-1大模型参数存储需求

图1-1 GPU内存容量增长落后于模型需求。

图1-2 单个NVIDIA DGX-2系统可用内存/存储的分布。相较GPU内存拥有3倍CPU内存和超过50倍的NVMe存储。

可以看出，随着参数规模、训练层数和隐藏层维度增长，单卡 GPU 显存容量逐渐成为训练系统瓶颈。为缓解该问题，业界和学术界提出了模型并行、流水线并行、张量并行、ZeRO 分片、Activation Checkpointing、CPU Offload 和 NVMe Offload 等技术。DeepSpeed ZeRO 系列通过分片优化器状态、梯度和参数降低单卡冗余显存占用；ZeRO-Infinity 进一步将 NVMe SSD 纳入 GPU、CPU、NVMe 组成的异构内存层级，说明 SSD 可作为大模型训练容量扩展的重要组成部分。

1.1.2 SSD 在训练系统中的角色变化

SSD 在大模型训练中的角色正在从“数据集存储介质”和“Checkpoint 保存介质”扩展为“训练状态容量扩展层”和“训练语义承载层”。传统训练流程中，SSD 主要用于保存训练语料和模型 Checkpoint，GPU 计算所需的中间状态主要驻留在 HBM 或 CPU DRAM 中。随着模型规模增长，CPU 内存和 SSD 被逐步纳入训练运行时，使训练系统能够在显存不足时继续运行更大模型或更长序列。

Smart-infinity研究把 Adam 等优化器的更新执行位置从 CPU 移到 SSD 内部 FPGA，实现近存运算，并加入Top-K 梯度压缩以节约流量。SSDTrain 研究分析得到activation 参数量增长速度快于其他参数，选择将其卸载到SSD，并将数据传输与计算完全重叠，在不影响性能的情况下降低 GPU 内存使用。这些工作共同说明，训练系统与 SSD 的关系已经从简单文件读写转向计算存储协同以及根据参数特性卸载。

图1-4 Smart-infinity架构图，将更新和解压缩计算卸载到SSD

图1-5 SSDTrain卸载和预取激活的流水线示意图

1.2 大模型训练框架与存储适配技术现状

1.2.1 PyTorch 训练框架现状

PyTorch 是大模型训练最主要的框架之一，其动态图机制、自动求导机制和丰富的 GPU 后端支持使其成为研究和工程训练系统的共同基础。PyTorch autograd 在前向传播过程中保存反向传播所需的中间张量，并在 backward 阶段读取这些保存张量计算梯度。这一机制使 PyTorch 成为观察 activation 生命周期的天然入口。

PyTorch 提供 module hook、parameter hook、torch.profiler、saved_tensors_hooks 等工具，能够在不改变模型数学定义的前提下采集训练过程中的张量元数据。通过 forward hook 可以获得模块输出 activation 的 shape、dtype、device 和大小；通过 gradient hook 可以获得参数梯度产生时间和规模；通过 saved_tensors_hooks 可以观察 autograd 保存和取回的中间张量；通过 optimizer state_dict 可以分析优化器状态结构；通过 profiler 可以记录 step 内算子、显存和时间开销。

PyTorch 原生 Tensor 管理主要关注设备间拷贝和自动求导正确性，并不会自动输出 SSD 写入粒度、生命周期标签或 AI SSD 语义接口。因此，本方案需要在 PyTorch 之上构建 Profiler、分类器和调度模块，将框架内部可见的训练语义提取出来，并转化为 SSD 友好的访问策略。

1.2.2 DeepSpeed 与 ZeRO 系列技术现状

DeepSpeed 是面向大模型训练的系统级优化框架，ZeRO 系列技术是当前具有代表性的训练状态分片机制。ZeRO-1 分片优化器状态，ZeRO-2 在此基础上分片梯度，ZeRO-3 进一步分片模型参数。通过逐步消除数据并行副本中的冗余状态，ZeRO 显著降低单卡显存压力。ZeRO-Offload 和 ZeRO-Infinity 将训练状态迁移至 CPU 或 NVMe，从而进一步扩大可训练模型规模。

图1-6 ZeRO系列原理图

ZeRO-Infinity 的重要意义在于证明 NVMe SSD 可以参与大模型训练运行时，而不仅是保存训练数据和 Checkpoint。它通过异构内存调度、数据预取和带宽重叠机制，在 GPU、CPU 和 NVMe 之间管理模型状态。现有 DeepSpeed Offload 配置通常以 offload device、nvme_path、buffer_count、buffer_size、max_in_cpu 等参数为核心，强调数据是否放入 NVMe 以及如何配置缓冲区，但没有直接表达“该写入属于 optimizer state”“该数据短生命周期”“该 activation 可重算”等训练语义。

本方案将在 DeepSpeed 默认 NVMe Offload 基础上分析其 I/O 模式，并增加语义分类、写入组织、调度优先级和元数据输出能力，使训练状态 Offload 不只是容量扩展机制，也成为可被 AI SSD 利用的语义化数据流。

1.3 现有方案不足与本方案切入点

现有训练框架和系统使用 SSD 的方式主要包括训练数据读取、Checkpoint 持久化和训练状态 Offload。DeepSpeed 支持 NVMe Offload，NVIDIA GPUDirect Storage 和 KvikIO 优化 GPU 与存储之间的数据通路，SSDTrain 和 FlashNeuron 关注 Activation Offload，BaM 和 G10 探索 GPU 与存储之间更直接或统一的访问架构。这些工作提供了重要基础，但多数方案仍以训练框架使用 SSD 为主，尚未解决训练框架如何适配 AI SSD 的问题。

不足主要体现在四个方面。第一，训练数据缺乏统一语义分类。参数、梯度、优化器状态、Activation、Checkpoint、数据集缓存和日志在训练框架内部是不同对象，但在存储层往往表现为普通文件读写。第二，Offload 策略主要关注显存和吞吐，对 SSD 写入粒度、生命周期隔离和写入削减关注不足。第三，多类 I/O 流之间缺少协调，Checkpoint 写入、Activation 临时写入、Optimizer State 更新和 Dataset Read 可能同时争用 SSD。第四，缺少面向 AI SSD 的接口规范，训练侧拥有语义但缺少稳定方式传递给底层。

2 研究目的及意义

2.1 研究目的

本方案的目标是建立一套从训练数据识别、生命周期建模、SSD 友好调度到 AI SSD 语义接口的完整方法。具体包括：构建面向 AI SSD 的训练数据语义分类模型；设计 SSD 友好的 Offload、预取、写回和 Checkpoint 调度方法；优化训练过程中的 I/O 访问模式；降低无效写入、小随机写和关键路径 I/O 等待；建立训练框架与 AI SSD 之间的语义协同接口机制。

这些目标共同服务于一个原则：在不改变模型结构和训练目标的情况下，让训练系统更可观测、更可调度、更适合 SSD。可重算 activation 优先重算，长期 optimizer state 采用分片和批量写回，checkpoint 采用异步分片顺序写，日志和 profile 采用缓冲与采样，数据集缓存采用预取和读缓存。

目标验证采用同环境 baseline 对比。对比对象包括 PyTorch 原生策略、DeepSpeed 默认 NVMe Offload 策略和本方案策略。评价指标包括 tokens/s、step time、GPU memory peak、Host writes、Small write ratio、Average write size、Checkpoint stall、Offload wait time 和 Tag coverage。

2.2 研究意义

本方案对大模型训练的意义在于提升训练系统资源使用的可控性。通过数据分类和调度，有限 GPU 显存可以承载更大模型、更长序列或更大 batch size；通过 SSD 友好 I/O 组织，训练系统可以降低因 Offload 和 Checkpoint 带来的吞吐损失；通过语义化调度，训练过程可以减少 I/O 随机性和抖动，提高长时间训练稳定性。

对 AI SSD 应用而言，训练侧输出的数据分类、生命周期提示和访问模式报告能够提供更接近真实训练负载的输入。相比通用 fio 或普通文件读写测试，训练语义 trace 能体现 activation、optimizer state、checkpoint、dataset cache 和 log/profile 等不同数据流的差异，为设备侧放置、调度、缓存和寿命相关优化提供依据。

对软硬件协同而言，该方案建立了训练算法侧与 AI SSD 侧之间的共同语言。训练侧说明数据含义、访问规律和调度意图；存储侧反馈设备统计、瓶颈现象和接口约束；双方基于统一 trace 和元数据迭代策略，使模型训练到存储设备之间形成端到端优化链路。

3 关键技术方案

3.1 总体技术路线

3.1.1 技术路线概述

总体技术路线采用“采集分析、语义建模、策略生成、框架适配、实验验证”的流程。首先，在 PyTorch 和 DeepSpeed 训练流程中插桩，采集参数、梯度、优化器状态、activation、checkpoint、dataset cache 和日志/Profile 数据的生命周期与 I/O 行为。其次，基于采集结果建立训练数据语义分类模型，形成数据类型、生命周期、访问模式、优先级、可重算性和持久化要求等标签。再次，根据分类结果设计 SSD 友好的 Offload、Prefetch、Write-back、Checkpoint 和日志管理策略。最后，将策略集成到训练框架中，并通过普通 NVMe SSD、软件 trace 和可用的 AI SSD 接口开展验证。在 AI SSD 接口明确后，可将训练语义标签映射到用户态库、驱动或设备接口；在联合测试中，可根据设备反馈调整调度阈值和 I/O 组织方式。

图3-1 总体技术路线图

3.1.2 总体架构

总体架构由七个模块组成：训练任务层、训练框架层、生命周期 Profiler、语义分类模块、SSD 友好调度模块、AI SSD 语义接口模块和实验评估模块。训练任务层提供模型、数据集和训练配置；训练框架层基于 PyTorch、DeepSpeed 和 ZeRO 执行训练；Profiler 负责采集元数据和事件；语义分类模块将对象转化为标签；调度模块决定 offload、prefetch、write-back 和写入组织；语义接口模块输出元数据或附带标签的 I/O 请求；评估模块统计训练性能和 I/O 指标。

表 7 给出模块职责。

模块	输入	输出	主要职责
训练任务层	模型、数据集、训练配置	训练事件	提供真实训练负载
训练框架层	PyTorch/DeepSpeed 运行时	Tensor 与状态对象	执行训练并暴露插桩点
生命周期 Profiler	Tensor 元数据、框架事件、I/O 事件	生命周期记录	采集创建、使用、释放、写回和大小信息
语义分类模块	生命周期记录、模型结构	数据标签	标注类型、生命周期、访问模式和优先级
SSD 友好调度模块	数据标签、资源状态	调度动作	决定 offload、prefetch、write-back 和降级策略
语义接口模块	数据标签、I/O 请求	元数据、trace、接口调用	向 AI SSD 侧传递训练语义
实验评估模块	训练日志、I/O 日志、设备统计	报告和图表	评估吞吐、显存、写入和稳定性

训练数据流从对象产生开始进入闭环。模型 forward 产生 activation，参数参与计算，optimizer state 在 optimizer step 中更新，checkpoint 在保存周期触发，dataset cache 在数据加载时被读取，日志/Profile 在训练过程中追加。Profiler 记录对象元数据和事件顺序，分类模块为对象生成标签，调度模块基于标签和资源状态选择策略，I/O 模块执行实际读写并输出 trace，评估模块再将结果反馈给分类阈值和调度策略。

闭环中包含两类反馈。第一类是训练侧反馈，例如显存峰值、step time、offload wait time、checkpoint stall 和 GPU utilization；第二类是存储侧反馈，例如 Host writes、请求大小分布、小写比例、队列深度和设备延迟。两类反馈共同决定后续策略是否需要调整。若 SSD 队列积压，则降低可延迟写入优先级；若显存峰值接近 OOM，则提高可安全 offload 数据的比例；若 checkpoint stall 明显，则调整 checkpoint 异步写和带宽限制。

图3-2 系统架构及数据流示意图

3.2 大模型训练数据生命周期分析技术

本节按照“数据类型识别、生命周期采集与特征建模、语义分类模型”的逻辑展开。数据类型识别用于明确训练过程中有哪些对象；生命周期采集与特征建模用于记录对象的产生、访问、迁移、写回和释放行为；语义分类模型则基于对象类型和生命周期特征生成标签、策略和聚合统计，为后续 SSD 友好调度提供输入。

图：生命周期分析技术路径图

3.2.1 训练数据类型识别

训练数据类型识别覆盖 Tensor 对象和文件对象两类数据。Tensor 对象主要包括模型参数、梯度、优化器状态和 Activation；文件对象主要包括 Checkpoint、Dataset Cache、日志文件和 Profile 文件。不同类型数据在产生阶段、生命周期、读写模式和存储需求方面存在明显差异，因此需要在采集阶段首先完成类型识别。

Tensor 对象主要通过 PyTorch 和 DeepSpeed 训练流程中的插桩进行识别。模型参数可通过 model.named_parameters()、DeepSpeed 参数管理状态或 ZeRO 分片信息识别；梯度可通过 backward 过程中的 parameter hook 或梯度分片状态识别；优化器状态可通过 optimizer 内部状态、Adam/AdamW 的一阶矩和二阶矩状态、ZeRO optimizer state 等信息识别；Activation 可通过模型 forward hook、autograd 保存张量事件等方式识别。通过这些方法，可以将训练框架内部的 Tensor 对象映射为 parameter、gradient、optimizer_state 和 activation 等基础数据类型。

文件对象主要通过训练框架回调、路径规则、文件命名规则、I/O 日志和系统监控进行识别。Checkpoint 通常由训练框架在固定 step 或 epoch 触发保存，可通过 checkpoint callback、保存路径和文件名识别；Dataset Cache 通常来自数据加载、预处理缓存或 mmap 数据文件，可通过数据集目录和 dataloader 行为识别；日志和 Profile 文件通常由 logger、TensorBoard、性能分析工具或 trace 工具产生，可通过文件路径、写入频率和事件来源进行识别。

表 3-2 给出训练数据类型识别对象。

数据类型	对象形态	主要来源	典型对象	基本特征
Parameter	Tensor	模型参数表、DeepSpeed 参数分片	weight、bias、embedding	长期存在，训练计算高频读取
Gradient	Tensor	backward 过程、parameter hook	.grad 、gradient shard	生命周期短，通常在 optimizer step 后释放或清零
Optimizer State	Tensor/State	optimizer 内部状态、ZeRO optimizer state	Adam exp_avg 、 exp_avg_sq 、 master weight	长期存在，周期读写，不可重算
Activation	Tensor	forward hook、autograd saved tensor	hidden states、attention output、MLP output	forward 产生，backward 使用，部分可重算
Checkpoint	File	checkpoint callback、保存路径	model state、optimizer state、training state	低频大块写，可靠持久化
Dataset Cache	File	dataloader、缓存目录、数据索引	tokenized shard、sample index、mmap file	读多写少，影响数据加载吞吐
Log/Profile	File	logger、profiler、trace 工具	event file、trace file、training log	高频小写，部分可采样或可丢弃

3.2.2 元数据记录与生命周期特征提取

在完成训练数据类型识别后，Profiler 需要进一步记录每个数据对象在训练过程中的产生、访问、迁移、写回和释放过程。该步骤把训练过程中分散在 PyTorch、DeepSpeed、优化器、Checkpoint 机制和系统 I/O 层的信息整理成可用于语义分类的基础输入。

生命周期采集需要覆盖多个层级。模型层通过 forward hook 采集模块输出 Activation；autograd 层通过 saved_tensors_hooks 采集中间张量保存和取回事件；参数层通过 parameter hook 采集梯度产生事件；优化器层采集 optimizer state 的初始化、访问和更新；DeepSpeed 层采集 ZeRO 分片参数、分片优化器状态、NVMe Offload buffer、prefetch 和 write-back 等状态迁移事件；系统 I/O 层采集文件写入、SSD 读写量、请求大小、队列深度和 I/O 延迟。

表 3-3 给出生命周期采集方法。

层级	采集对象	采集方法	采集信息
PyTorch Module	activation	forward hook	shape、dtype、size、layer、device
Autograd	saved tensor	saved_tensors_hooks	保存时间、取回时间、大小、重用距离
Parameter	gradient	parameter hook	梯度产生时间、大小、所属参数
Optimizer	optimizer state	state_dict 或内部状态	m/v/master weight 大小和更新窗口
DeepSpeed ZeRO	shard state	DeepSpeed engine 状态	分片参数、offload buffer、prefetch 事件
Checkpoint	状态文件	framework callback	文件大小、写入时间、版本信息
系统 I/O	SSD 请求	iostat、nvme-cli、eBPF 或日志	读写量、请求大小、延迟、队列深度

为了统一描述不同来源的数据对象，建立标准化元数据结构。元数据主要包括三类：第一类是对象身份信息，用于说明该对象是谁、属于哪类数据、来自哪个层和哪个 rank；第二类是规模与位置描述，用于说明该对象的数据大小、数据类型和当前所在设备；第三类是底层存储与视图关系，用于处理 Tensor 别名、view、slice、reshape、transpose 等情况，避免重复统计和重复写入 SSD。

表 3-4 给出训练数据对象统一元数据字段。

字段	类型	含义	示例
object_id	对象身份字段	训练数据对象的唯一编号	rank0_step10_layer3_q
name	对象身份字段	数据对象名称，通常来自模型模块名、参数名或文件名	transformer.layers.3.attn.q
data_type	对象身份字段	数据类型	parameter 、 gradient 、 activation 、 optimizer_state 、 checkpoint
layer_id	对象身份字段	所属模型层编号	3
rank_id	对象身份字段	分布式训练中的进程编号	rank0
shape	规模与位置字段	Tensor 的逻辑形状	[8, 16, 128, 64]
dtype	规模与位置字段	数据精度类型	fp32 、 fp16 、 bf16
size_bytes	规模与位置字段	数据大小，单位为字节	16777216
device	规模与位置字段	当前所在设备或存储位置	cuda:0 、 cpu 、 nvme
created_at	事件字段	数据产生的时间或训练阶段	step10_forward_layer3
last_used_at	事件字段	数据最后一次被使用的时间或训练阶段	step10_backward_layer3
read_count	事件字段	数据被读取的次数	1
write_count	事件字段	数据被写入的次数	1
recomputable	策略辅助字段	是否可以通过重新计算得到	true 、 false
persistence	策略辅助字段	数据是否需要持久保存	temporary 、 long_lived 、 persistent
storage_id	底层存储字段	底层 storage 的唯一标识	cuda0_ptr140390129344512_size50331648
storage_offset	底层存储字段	Tensor 在底层 storage 中的起始偏移	0 、 1024
stride	底层存储字段	Tensor 在 storage 中按各维访问的步长	[131072, 8192, 64, 1]
is_view	底层存储字段	是否是其他 Tensor 的视图	true 、 false
base_object_id	底层存储字段	如果当前对象是视图，指向其基础对象	rank0_step10_layer3_qkv
alias_group_id	底层存储字段	共享同一底层 storage 的对象组编号	alias_group_003

在元数据基础上，生命周期特征提取进一步把训练事件转化为少量可用于分类的行为特征。对于所有对象，重点提取数据规模、生命周期长度、重用距离、读写次数和写入频率；对于不同数据类型，再提取少量类型相关特征。例如，Activation 重点关注 backward 重用距离和可重算性；Optimizer State 重点关

注更新频率；Checkpoint 重点关注保存周期和写入规模；Dataset Cache 重点关注读取频率和缓存命中情况；Log/Profile 重点关注事件频率和可丢弃性。

表 3-5 给出生命周期特征提取内容。

特征	适用对象	含义
size_bytes	全部对象	数据对象大小
lifetime	全部对象	数据从产生到最后使用或释放的时间跨度
reuse_distance	Tensor 对象	数据从产生或上次使用到下次使用之间的距离
read_count	全部对象	数据读取次数
write_count	全部对象	数据写入次数
write_frequency	全部对象	单位 step 或单位时间内的写入频率
backward_reuse_distance	Activation	forward 产生到 backward 使用之间的距离
update_frequency	Optimizer State	optimizer step 中被更新的频率
save_interval	Checkpoint	Checkpoint 保存周期
cache_hit_rate	Dataset Cache	缓存命中率
event_frequency	Log/Profile	日志或 Profile 事件产生频率
discardability	Log/Profile	数据是否可采样、延迟或丢弃

3.2.3 训练数据语义分类模型

本项目的语义分类模型拟采用“规则驱动 + Profiler 反馈修正”的方式构建。项目初期首先使用可解释规则，以 3.2.1 中识别出的数据类型和 3.2.2 中采集到的生命周期特征为输入，生成面向调度模块和 AI SSD 语义接口的分类标签，对训练数据进行分类，以降低实现复杂度并便于调试。例如，Checkpoint 具有低频、大块、可靠持久化特征，可默认标注为 persistent、append_only 和 large_write；Activation 通常在 forward 阶段产生并在 backward 阶段使用，可默认标注为 short_lived 和 write_once_read_once，并进一步根据是否可重算、大小和重用距离决定是否写入 SSD；Optimizer State 贯穿整个训练过程，并在 optimizer step 中周期性读写，可默认标注为 long_lived、read_write 和 periodic_update；Dataset Cache 主要服务于数据读取，可默认标注为 read_mostly。

在规则运行后，Profiler 会根据实际训练过程中的统计结果修正分类阈值。可用于反馈的指标包括 Activation 大小分布、backward 重用距离、Optimizer State 更新频率、Checkpoint 写入时间、SSD 带宽利用率、GPU 显存压力和关键路径等待时间等。例如，当 GPU 显存峰值接近 OOM 时，可降低安全 Offload 阈值或提高 Activation 重计算比例；当 SSD 队列积压或写入延迟升高时，可提高 Activation Offload 阈值，并降低日志/Profile 等可延迟数据的写入优先级；当 Checkpoint stall 明显时，可调整 Checkpoint 异步写入和带宽限制。通过这种反馈机制，分类规则能够逐步适配具体模型规模、batch size、sequence length、GPU 显存容量和 SSD 性能条件。

表 9 给出规则分类与策略映射。

数据类别	生命周期标签	访问模式标签	默认策略	SSD 友好组织方式
Parameter	long_lived	read_heavy/read_write	分层缓存、按需预取	layer-wise shard
Gradient	short_lived	write_then_read	优先 GPU/CPU 管理	谨慎 offload，聚合后写
Optimizer State	long_lived	periodic_read_write	分片 offload	fixed-size shard、批量写回
Activation	short_lived	write_once_read_once	重算优先，选择性 offload	ring buffer、预分配 slot
Checkpoint	persistent	append_only	异步分片保存	大块顺序写、版本管理
Dataset Cache	persistent/read_mostly	read_heavy	预取与缓存	大文件 shard、mmap
Log/Profile	temporary/discardable	small_write	缓冲与采样	批量追加、低优先级

分类模块输出应包括单对象标签和聚合统计。单对象标签用于调度，聚合统计用于报告和策略评估。聚合统计包括每类数据大小占比、进入 SSD 的比例、平均生命周期、平均重用距离、写入次数、写入块大小分布和关键路径等待贡献。

3.3 SSD 友好的训练数据 Offload 策略

3.3.1 参数数据 Offload 策略

参数数据 Offload 需要与模型执行顺序紧密结合。Transformer 模型通常按 layer 顺序执行 forward 和 backward，ZeRO-3 在使用参数前 gather，在使用完成后释放或重新分片。方案利用这一规律设计 layer-wise 参数预取，使后续 layer 所需参数在计算到达前进入 GPU 或 CPU 缓存，降低 GPU 等待时间。

参数策略综合考虑显存压力、CPU 缓存容量、SSD 读取带宽和 layer 执行时间。如果显存压力较低，更多参数可驻留 GPU，减少 SSD 读取；如果显存压力较高，则采用更积极的释放和预取策略。对频繁访问的热参数，优先保存在 CPU 或 GPU；对冷参数分片，保存在 SSD 并按需读取。训练侧标注参数数据为 long_lived、read_heavy 或 read_write，使其区别于短生命周期 activation。

参数预取窗口可根据 layer 计算时间和 I/O 延迟估算。若下一层参数读取预计耗时小于当前层计算时间，可在当前层计算开始后异步预取；若读取耗时较长，则需要扩大预取提前量或提高缓存驻留比例。若 SSD 负载过高，调度器可优先保证即将使用的参数分片，推迟日志、profile 或可重算 activation 的写入。

3.3.2 优化器状态 Offload 策略

优化器状态是 SSD Offload 的重要对象。Adam/AdamW 优化器状态规模大、长期存在、周期读写，适合分片管理，但不适合以大量小 Tensor 文件形式写入 SSD。方案采用固定大小 shard 组织优化器状态，将多个小状态合并为较大连续块，并按 rank、layer 或参数范围管理偏移。这样既减少文件数量和元数据开销，也便于 AI SSD 识别其为长期、周期更新数据。

优化器状态写回采用批量化和异步化机制。在 optimizer step 中，训练框架优先处理当前更新窗口所需状态，其他状态根据依赖关系延迟写回。若 CPU 内存允许，可将近期状态保留在 CPU 缓存中，避免每 step 都完整写回 SSD。对于必须写回的数据，采用大块连续写或日志式追加，减少小随机写。

Fixed shard 布局需要维护稳定索引。每个 shard 记录 rank_id、param_range、state_type、offset、size_bytes、version 和 checksum。读取时根据参数范围定位对应状态块，写回时更新版本和校验。若采用日志式追加，则需要维护新旧版本映射和回收机制；若采用固定 offset 更新，则需要关注覆盖写热点。两种方式可根据设备能力和实验结果选择。

3.3.3 激活值管理策略

Activation 管理是 SSD 适配中最需要精细决策的部分。Activation 生命周期短，若不加选择地写入 SSD，可能产生大量临时写入。方案采用“重算优先、选择性 Offload、缓冲复用”的策略。可重算且计算代价相对较低的 activation 使用 checkpointing 或 recomputation；不可重算、尺寸较大、反向重用距离较长且显存压力明显的才写入 SSD。

Activation 写入 SSD 时采用 ring buffer。该方式通过预分配固定大小文件或区域，将 activation 按 slot 写入和复用，避免每个 activation 创建独立文件。反向传播读取后，slot 立即标记为空并复用。每个 slot 记录 tensor_id、layer_id、microbatch_id、size_bytes、write_time、read_time 和 valid_flag。这样可以降低文件系统元数据开销，并使短生命周期数据与 checkpoint、optimizer state 等长期数据隔离。

Activation 调度需要处理流水线并行和梯度累积场景。不同 microbatch 的 activation 生命周期可能交错，ring buffer 需要按 microbatch_id 和 layer_id 管理 slot，避免覆盖尚未被 backward 使用的数据。对于后几层 activation，由于 forward 后很快进入 backward，可直接保留在 GPU 或 CPU，不必写入 SSD。SSDTrain 的研究也表明，调度器需要知道微批阶段和即将到来的 backward 顺序，才能提前预取 activation。

3.3.4 梯度数据管理策略

梯度通常不作为 SSD Offload 的优先对象。梯度生命周期短，并与 backward、通信和 optimizer step 高度耦合。若大量梯度写入 SSD，容易增加关键路径 I/O。默认策略是梯度优先保留在 GPU 或 CPU 中完成聚合、reduce-scatter 和更新，避免将其作为常规 SSD 写入对象。

在极端显存压力或较大 gradient accumulation 场景下，可探索梯度分片临时写入 SSD，但必须满足两个条件。第一，写入粒度足够大，不能逐小 Tensor 写入；第二，读取时机可预测，避免 optimizer step 等待 SSD。梯度 Offload 应作为可选增强策略，并通过 profiler 证明其收益大于 I/O 代价后才启用。

梯度相关 trace 仍需采集。即使不写入 SSD，梯度产生时间、大小、通信时间和释放时间也影响 optimizer state 调度和显存压力判断。梯度生命周期信息可用于判断 activation 是否需要 offload、参数是否可释放以及 optimizer state 是否应提前预取。

3.3.5 Checkpoint 优化策略

Checkpoint 优化重点解决突发写入和训练阻塞问题。方案支持分片 Checkpoint，使不同 rank 并行保存自身状态，避免单点聚合；支持异步 Checkpoint，使写入尽可能脱离训练关键路径；支持大块顺序写，使 SSD 更高效处理持久化数据；支持版本管理，避免保留过多历史状态造成空间和写入压力。

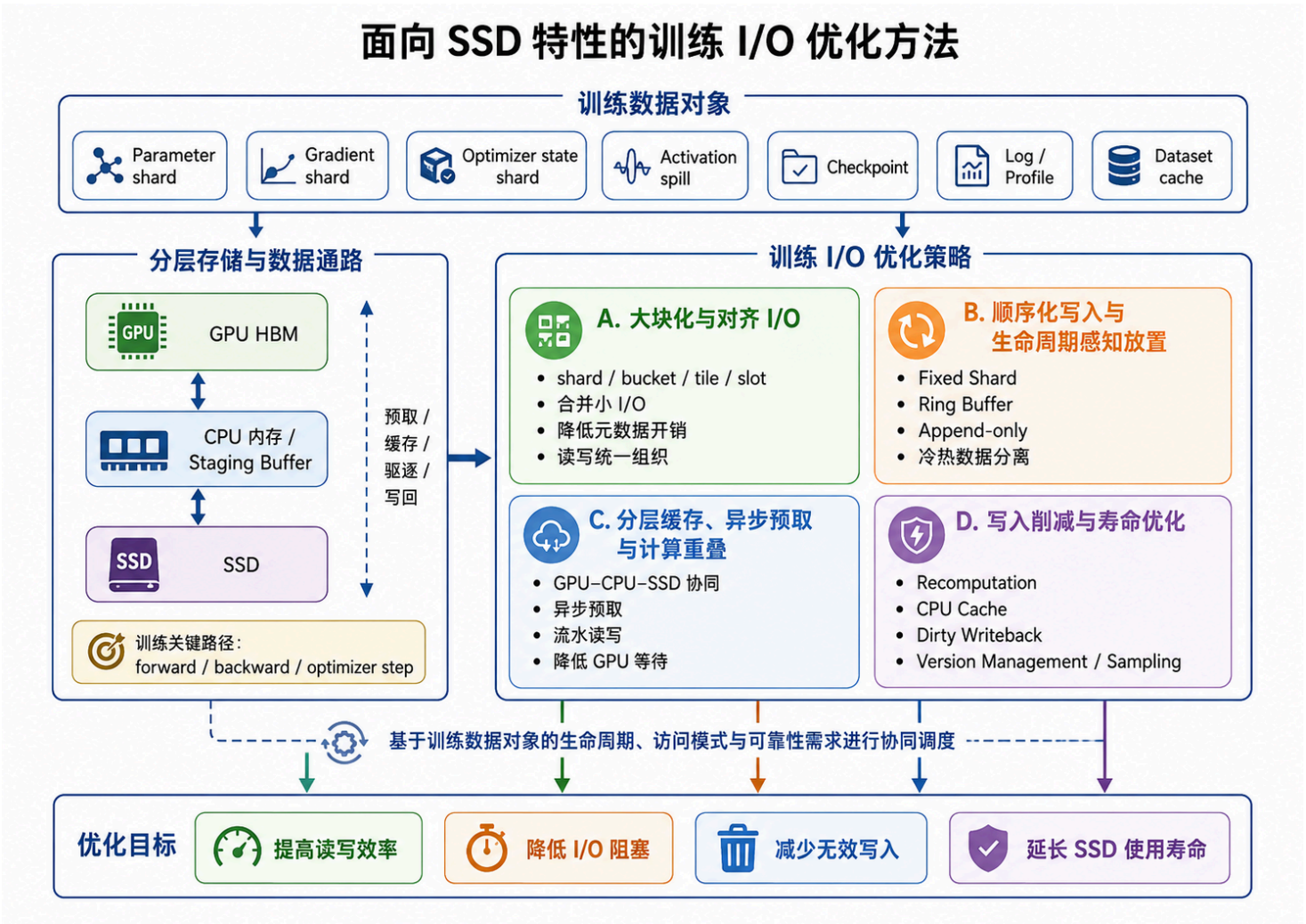
Checkpoint 可靠性不能因写入优化而削弱。每个 checkpoint 版本应包含元数据、状态分片、完整性校验、训练步数、随机数状态和恢复所需的分布式拓扑信息。异步写入完成前，应清晰标记版本状态，避免训练中断后误用未完成 checkpoint。恢复测试应成为实验评估的一部分。

Checkpoint 与 Offload 流需要解耦。若 checkpoint 写入与 optimizer state 写回或 activation offload 同时发生，会形成 I/O 峰值。调度器应在 checkpoint 写入期间降低日志写入优先级，限制可延迟 offload，必要时调整 checkpoint 带宽上限。训练侧应标注 checkpoint 为 persistent、append_only、reliable 和 low_frequency_large_write，帮助 AI SSD 区分其与临时数据。

3.4 面向 SSD 特性的训练 I/O 优化方法

在大模型训练中，模型参数、梯度、优化器状态、activation、checkpoint、日志和性能分析数据会产生大量读写需求。随着模型规模扩大，GPU 显存难以容纳全部训练状态，需要引入 CPU 内存和 SSD 构成分层存储结构，将部分数据卸载到主机侧存储中。SSD 容量大、顺序读写带宽较高，适合承担训练状态的扩展存储任务；但其访问延迟高于 GPU 显存和 CPU 内存，并且对小粒度、随机、频繁覆盖写较为敏感。如果训练系统直接以单个 Tensor 为单位频繁读写 SSD，容易产生大量小 I/O、随机 I/O 和元数据开销，从而影响训练吞吐。

因此，面向 SSD 的训练 I/O 优化需要结合训练数据的语义进行设计。不同数据对象具有不同生命周期、访问模式和可靠性要求。例如，参数分片和优化器状态会直接影响训练主流程，需要按计算顺序及时预取和写回；activation 生命周期较短，可通过重算减少保存；checkpoint 数据量大但写入频率较低，适合后台顺序写入；日志和性能分析数据主要用于诊断，应采用低优先级缓冲追加。基于这些差异，系统需要对 SSD I/O 进行对象感知的组织、调度和削减。



数据对象	生命周期	访问特征	是否影响训练关键路径	SSD 优化重点
日志/性能分析数据	短到中期	追加写，诊断时读取	否	缓冲追加、采样记录、低优先级处理
数据集缓存	长期	读密集	间接影响训练	与写密集卸载数据分离

通过上述分类，系统可以避免将所有数据都作为普通文件处理，而是根据数据的训练语义采用差异化 I/O 策略，从而提高 SSD 读写效率，减少无效写入和空间回收压力。

3.4.1 大块化与对齐 I/O 方法

大块化 I/O 是降低 SSD 小粒度访问开销的基本方法。训练过程中存在大量尺寸不同的 Tensor。如果系统按单个 Tensor 粒度向 SSD 提交读写请求，会产生大量小 I/O，带来系统调用、文件元数据维护、I/O 队列调度和 SSD 内部映射管理等开销。这些开销最终可能转化为 GPU 等待时间，影响训练吞吐。

因此，系统应将零散 Tensor 组织为更大的逻辑 I/O 单元。参数分片可以按层或计算节点合并为预取批次；优化器状态可以按分片或分块组织为批量读写单元；activation 临时数据可以写入可复用槽位；checkpoint 可以按计算节点和分片组织为较大的文件；日志和性能分析数据可以按时间窗口缓冲追加。这样，SSD 侧看到的是数量更少、粒度更大、结构更清晰的 I/O 请求。

大块化 I/O 不仅适用于写入，也适用于读取和预取。对于参数分片，系统可以根据前向和反向传播顺序，提前将后续层所需数据读取到 CPU 缓冲区；对于优化器状态，系统可以在参数更新阶段以分块为单位批量读取、更新和写回；对于 checkpoint，系统可以将多个小状态合并后统一写入，减少小文件数量和频繁元数据更新。

大块化的关键是使 I/O 单元与训练调度单元一致。若只是为了增大 I/O 块而盲目合并数据，可能导致读放大或缓存浪费。例如，将短期不会使用的参数与即将使用的参数放在同一个 I/O 块中，虽然单次读取变大，但会占用 CPU 缓冲区，并可能挤占真正需要的数据。因此，参数合并应服务于层执行顺序，优化器状态合并应服务于参数更新顺序，activation 组织应服务于反向传播的重用顺序。

为支持上述机制，系统需要维护卸载对象元数据，包括对象类型、所属计算节点、层编号、分片编号、数据大小、生命周期、访问模式、重用距离和可靠性需求等信息。调度器据此判断哪些对象可以合并，哪些对象需要提前读取，哪些对象可以延迟写回，哪些对象可以释放或重算。由此，大块化不只是存储层优化，也成为训练运行时数据管理的一部分。

大块化阈值需要根据训练任务和系统资源动态调整。阈值过小，难以减少小 I/O；阈值过大，则可能增加等待时间和 CPU 缓冲区占用。对于训练关键路径上的参数分片和优化器状态，应避免因过度合并而延误计算；对于 checkpoint、日志和性能分析数据等后台数据，则可以采用更大的缓冲区和更长的合并窗口。

3.4.2 顺序化写入与生命周期感知放置方法

SSD 更适合处理连续、大块、生命周期相近的数据流。虽然 SSD 的随机访问性能优于机械硬盘，但频繁覆盖、小粒度、地址分散的写入仍会增加内部管理压力。若训练系统将短生命周期 activation、频繁更新的优化器状态、长期保存的 checkpoint 和低优先级日志混合写入同一逻辑区域，就容易造成冷热数据混放和额外空间回收开销。

因此，系统应结合顺序化写入和生命周期感知放置。顺序化写入关注单个数据流的组织方式，生命周期感知放置关注不同数据流之间是否应分离。系统可将训练数据划分为长期训练状态、短期临时数据、后台可靠性数据、低优先级诊断数据和读密集缓存数据，并为每类数据设计不同的 SSD 放置策略。

对于参数分片和优化器状态分片，这类数据属于长期训练状态，需要稳定索引，便于快速定位和恢复。参数分片可以按计算节点、层或分片编号组织，支持按计算顺序预取。优化器状态可以采用固定分片方式，为每个状态分片分配稳定位置，并在参数更新阶段按分块进行大块读写。该方式索引清晰、恢复方便，但需要合理管理更新偏移和写回时机，避免过多小范围覆盖写。

对于 activation 临时数据，其生命周期通常只覆盖前向传播之后到反向传播使用之前。系统可以采用预分配连续区域和环形槽位管理。当某个槽位中的 activation 完成反向传播使用后，该槽位即可复用。这样既避免频繁创建和删除小文件，也便于快速回收临时空间。需要注意的是，将 activation 写入 SSD 应作为补充策略。系统应优先采用重算或 CPU 卸载，只有当 CPU 内存不足或 activation 压力过大时，才将部分数据临时写入 SSD。

对于 checkpoint，推荐采用追加写或分片顺序写方式。每次保存 checkpoint 时，系统按计算节点和分片生成连续的大文件或大块区域，而不是对大量小文件进行随机覆盖。旧 checkpoint 的删除和版本清理应由后台任务完成，避免阻塞训练主流程。由于 checkpoint 通常不在每个训练步的关键路径上，其写入应作为低优先级后台 I/O，与参数预取和优化器状态读写错峰执行。

对于日志和性能分析数据，应采用缓冲追加方式。正常训练阶段只记录关键事件和摘要信息；当出现异常、性能下降或调试需求时，再提高记录频率。对于数据集缓存，由于其以读取为主，应与优化器状态、activation 临时数据等写密集区域分离，避免读写互相干扰。

表 11 对比了不同写入组织方式及其适用场景。

表 11 不同写入组织方式对比

写入组织方式	适用数据	优点	风险	使用建议
每 Tensor 独立写	小型调试数据	实现简单	小文件多、随机性强、元数据开销高	不作为主要策略
固定分片	参数分片、优化器状态	索引稳定、恢复方便	需要管理固定偏移和版本	重点采用
环形缓冲	activation 临时数据、临时梯度数据	适合短生命周期复用	需要管理槽位生命周期	有条件采用
追加写	checkpoint、日志/性能分析数据	顺序性好、实现清晰	需要版本清理机制	重点采用
日志式写入	频繁更新状态	顺序写友好	需要后台整理和索引维护	作为增强探索
读密集缓存	数据集缓存	有利于读取稳定性	可能与写密集数据竞争空间	与写密集区域分离

如果 SSD 支持主机侧放置提示、多流写入、区域化写入或类似接口，训练框架可以进一步将数据对象的生命周期和访问模式传递给设备侧。例如，将 activation 标记为短生命周期临时数据，将 checkpoint 标记为长期顺序写数据，将优化器状态标记为频繁更新数据，从而帮助 SSD 更合理地组织内部空间。

3.4.3 分层缓存、异步预取与计算 I/O 重叠方法

在 SSD 协同卸载训练中，影响训练吞吐的关键不只是 SSD 的裸读写速度，而是 SSD I/O 是否暴露在 GPU 计算关键路径上。如果 GPU 在前向传播、反向传播或参数更新阶段等待 SSD 数据返回，即使 SSD 本身带宽较高，也会导致训练吞吐下降。因此，系统需要通过分层缓存、异步预取和流水调度，将 SSD I/O 尽可能隐藏在 GPU 计算过程中。

本方案采用 GPU 显存、CPU 内存和 SSD 构成的三级数据管理结构。GPU 显存保存当前计算立即需要的数据；CPU 内存作为中间缓存和数据暂存区，承担 SSD 与 GPU 之间的数据中转；SSD 保存暂时无法驻留在 GPU 或 CPU 中的参数分片、优化器状态、activation 临时数据和 checkpoint。训练调度器根据数据访问顺序，在合适时间将数据从 SSD 预取到 CPU，再传输到 GPU，同时将暂时不再需要的数据从 GPU 或 CPU 驱逐到下一级存储。

在前向传播阶段，调度器根据层执行顺序提前预取后续层的参数分片。当 GPU 正在计算当前层时，系统可以异步读取未来层所需数据，并放入 CPU 暂存区或进一步传输到 GPU。这样，参数读取可以与当前层计算重叠，减少 GPU 等待。

在反向传播阶段，调度器需要同时考虑参数、activation 和梯度的生命周期。反向传播通常与前向传播顺序相反，因此系统可以根据反向顺序预取即将使用的 activation 或参数分片。已经完成梯度计算且后续不再需要的 activation 可以释放、标记为可重算，或在必要时写入临时区域。梯度分片应优先在通信和参数更新中直接使用，只有在内存压力较大时才考虑短期写入 SSD。

在参数更新阶段，优化器状态是 SSD 卸载中的重要对象。以 Adam 类优化器为例，每个参数分片通常对应一阶矩和二阶矩等状态，数据量较大。系统可以将优化器状态按分块组织，在处理当前分块的同时读取下一分块，并将已更新完成的上一分块异步写回 SSD，形成“读下一块、算当前块、写上一块”的流水结构。该方式可以避免参数更新阶段被大规模读写完全阻塞。

为了实现有效重叠，系统需要维护异步 I/O 队列和 CPU 缓冲池。异步 I/O 队列负责向 SSD 提交并发读写请求；CPU 缓冲池负责承接 SSD 与 GPU 之间的数据搬运；训练调度器负责决定预取距离、驱逐时机和写回顺序。当 CPU 缓冲区紧张时，应优先保留即将使用的数据，驱逐重用距离较远的数据；当 checkpoint 或日志写入与关键路径 I/O 冲突时，应降低其优先级或延迟执行。

此外，系统还需要避免过度预取。预取距离过短，数据可能无法及时到达 GPU；预取距离过长，则会占用 CPU 缓冲区，并可能导致数据在真正使用前被挤出缓存。因此，调度器应根据层执行顺序、数据重用距离、CPU 缓冲区占用和当前 I/O 队列状态动态调整预取策略。

3.4.4 写入削减与 SSD 寿命优化方法

写入削减是降低 SSD 压力和延长设备使用寿命的重要方法。对于大模型训练而言，并非所有数据都必须完整写入 SSD。不同训练数据具有不同的可靠性要求：模型参数和优化器状态属于核心训练状态，必须保证一致性；activation 多数情况下可以通过重算获得；日志和性能分析数据主要用于诊断，可以降低记录频率；checkpoint 虽然重要，但可以通过保存策略和版本管理控制写入规模。因此，写入削减应基于训练语义进行，而不能简单丢弃数据。

第一类方法是通过重算减少 activation 写入。Activation 在反向传播中需要使用，但并不一定全部保存。系统可以只保存少量关键中间结果，在反向传播阶段通过重新执行部分前向计算恢复所需 activation。这样可以减少 activation 对 GPU 显存、CPU 内存和 SSD 的存储压力。对于必须保存但使用周期较短的 activation，可以优先放入 CPU 内存；只有当 CPU 内存不足或重算代价过高时，才将其临时写入 SSD。

第二类方法是通过 CPU 缓存和脏数据写回减少优化器状态的频繁写入。优化器状态在每个参数更新阶段都会变化，如果每次更新后立即写回 SSD，容易造成大量重复写入。系统可以在 CPU 内存中缓存近期可能继续访问的优化器状态分片，并通过脏标记记录其是否被修改。当某个分片短期内仍可能被使用时，可以暂缓写回；当内存压力增大或进入合适的后台写回阶段时，再将多个已修改分片批量写入 SSD。

第三类方法是通过 checkpoint 策略控制可靠性数据写入。Checkpoint 是训练恢复的重要保障，不能随意省略，但也不需要过于频繁地进行全量保存。系统可以根据训练阶段、迭代间隔、验证结果或故障恢复需求决定保存频率，并通过版本管理控制历史 checkpoint 数量。对于长期训练任务，可以保留最近版本和关键里程碑版本，删除过旧且价值较低的版本。若系统支持增量保存，也可以只保存相对上一版本发生变化的数据，但需要保证恢复流程正确。

第四类方法是减少低价值日志和性能分析数据写入。日志和性能分析数据对调试有帮助，但不应在正常训练阶段以过高频率写入 SSD。系统可以采用采样、缓冲和分级记录机制：正常阶段只记录关键事件和摘要信息；当训练出现异常、吞吐下降或数值不稳定时，再提高记录频率并保存更详细信息。

第五类方法是小 Tensor 写入合并。对于仍然需要写入 SSD 的小规模数据对象，系统不应逐个提交写入请求，而应先在内存中合并为更大的写入块。该方法可用于小规模状态、日志片段、调试信息以及部分临时数据。合并过程需要记录每个对象在大块中的偏移和长度，以便恢复或读取时正确定位。

表 12 总结了主要写入削减策略。

表 12 写入削减策略

策略	适用对象	削减逻辑	可能代价	使用建议
重算	activation	用计算替代保存	增加 GPU 计算	优先采用
CPU 缓存	优化器状态	合并多次更新后批量写回	增加 CPU 内存占用	重点采用
脏数据写回	优化器状态、参数分片	只写回被修改的数据块	需要维护脏标记	重点采用
版本管理	checkpoint	控制保存频率和保留版本	历史恢复点减少	重点采用
采样记录	日志/性能分析数据	降低记录频率	诊断粒度下降	常规采用
写入合并	小 Tensor、小日志	合并后写入	增加缓冲管理复杂度	常规采用

需要强调的是，写入削减不能破坏训练正确性和恢复能力。模型参数、优化器状态、随机数状态、学习率调度器状态等关键数据必须在 checkpoint 中保持一致；activation 重算必须保证反向传播结果正确；日志采样也应保留必要的错误信息和诊断线索。系统应为每类削减策略设置明确边界，确保优化 SSD I/O 的同时不影响训练可靠性。

3.5 面向 AI SSD 的训练语义提示接口

面向 AI SSD 的训练语义提示接口设计

训练系统—AI SSD 协同的数据语义建模、标签传递与接口实现方案

1 接口设计原则与语义模型

训练框架 / 训练系统

PyTorch
DeepSpeed / ZeRO
Activation 管理
Checkpoint 管理
Profiler / 调度模块

训练语义提示接口

API

AI SSD / 存储管理组件

数据放置与分层
缓存管理
调度与资源分配
寿命管理与维护
性能与能效优化

传递训练侧语义信息

驱动存储侧优化决策

目标：将训练框架内部已掌握的数据含义传递给 AI SSD 或相关存储管理组件

作用：使存储系统感知训练数据的生命周期、访问模式、重要程度

用途：支撑数据放置、缓存管理、调度优化和资源分配

原则：可解释、可扩展、可验证、与训练框架解耦

统一语义对象 = 数据类型 + 生命周期 + 访问模式 + 优先级 + 可靠性 + 持久化要求

典型数据类型（示例）

Parameter Gradient Optimizer State Activation
Checkpoint Dataset Cache Log / Profile

① Activation：短生命周期、写一次读一次；Checkpoint：持久化、大规模顺序写

2 语义标签体系与接口字段设计

数据类型标签	Parameter / Optimizer State / Activation / Checkpoint 等
生命周期标签	temporary、short-lived、long-lived、persistent
访问模式标签	read-heavy、write-heavy、read-write、append-only、write-once-read-once
优先级标签	critical、high、normal、low、background
可靠性 / 持久化要求	persistence、checksum 等

每个数据对象对应统一描述结构：必选字段 + 扩展字段

3 核心接口字段结构（示例）

字段	含义	示例	字段	含义	示例
object_id	数据对象唯一标识	rank0_layer12_opt_shard3	persistence	是否持久化	true
data_type	数据类型	optimizer_state	rank_id	分布式 rank	0
lifecycle	生命周期	long_lived	layer_id	模型层编号	12
access_pattern	访问模式	periodic_read_write	reuse_distance	重用距离	backward_layer_gap=8
priority	优先级	critical	suggested_io	建议 I/O 形态	fixed_shard
size_bytes	数据大小	268433456	version	数据版本	step_10000
recomputable	是否可重算	false	checksum	完整性校验	sha256:...

4 接口实现与协同机制

阶段一：目录组织规则 + Sidecar Metadata + I/O Trace

输出训练语义信息

支持高层验证

不同类型数据独立存放并生成元数据记录

阶段二：用户态库 / 训练框架插件

在实际 I/O 请求发起时附带语义标签

实现语义与读写请求同步传递

阶段三：映射至驱动层或 AI SSD 专用接口

与设备能力对接

实现训练系统与 AI SSD 的深度协同

闭环协同机制

训练侧

提供数据类型、生命周期、访问模式、优先级

存储侧

开展数据放置、缓存管理、调度优化、寿命管理

闭环

统一元数据格式 + Trace 分析 + 反馈优化

5 接口价值 / 设计原则

可解释

语义信息清晰可读

可扩展

兼容多类训练对象与未来能力

可验证

支持元数据与 Trace 验证

轻量解耦

不改变训练数据内容

协同优化

支持软硬件闭环联动

标准化

便于形成统一接口规范

设计目标：构建统一、语义化、可扩展的训练系统—AI SSD 协同接口，为数据放置、缓存管理、调度优化与资源分配提供基础支撑。

3.5.1 接口设计原则与语义模型

训练语义提示接口的目标是将训练框架内部已经掌握的数据含义传递给 AI SSD 或相关存储管理组件，使存储系统能够感知训练数据的生命周期、访问模式和重要程度，从而为数据放置、缓存管理、调度优化和资源分配提供依据。

本接口不依赖特定 SSD 硬件实现，也不改变训练数据本身内容，仅负责传递训练侧已经掌握的语义信息。接口设计遵循可解释、可扩展、可验证和与训练框架解耦的原则，既能够通过元数据文件和离线 Trace 实现验证，也能够进一步扩展到用户态库、驱动程序或 AI SSD 专用接口。

在语义模型方面，训练数据被统一描述为具有数据类型、生命周期、访问模式、优先级、可重算性和持久化要求等属性的语义对象。典型数据类型包括 Parameter、Gradient、Optimizer State、Activation、Checkpoint、Dataset Cache 和 Log/Profile。不同数据对象具有不同的生命周期和访问特征，例如 Activation 通常属于短生命周期且写一次读一次的数据，而 Checkpoint 则属于持久化的大规模顺序写数据。通过统一语义模型，可为后续调度和存储优化提供标准化描述。

3.5.2 语义标签体系与接口字段设计

为了准确描述训练数据特征，接口定义了一套统一语义标签体系。标签内容主要包括数据类型、生命周期、访问模式、优先级以及可靠性要求等信息。

数据类型标签用于区分 Parameter、Optimizer State、Activation、Checkpoint 等不同训练对象；生命周期标签用于描述 temporary、short-lived、long-lived 和 persistent 等生命周期特征；访问模式标签用于表示 read-heavy、write-heavy、read-write、append-only、write-once-read-once 等访问行为；优先级标签用于区分 critical、high、normal、low 和 background 等不同等级的数据流。

在接口实现中，每个数据对象对应一个统一描述结构。核心字段包括 object_id、data_type、lifecycle、access_pattern、priority、size_bytes 和 persistence 等必选字段；扩展字段包括 recomputable、rank_id、layer_id、reuse_distance、suggested_io、version 和 checksum 等信息。

通过统一字段定义，训练框架能够持续输出结构化语义信息，为 AI SSD 提供可解析、可统计和可利用的数据依据。

表 12 给出接口字段建议。

字段	含义	示例
object_id	数据对象唯一标识	rank0_layer12_opt_shard3
data_type	数据类型	optimizer_state
lifecycle	生命周期	long_lived
access_pattern	访问模式	periodic_read_write
priority	优先级	critical
size_bytes	数据大小	268435456
recomputable	是否可重算	false
persistence	是否持久化	true
rank_id	分布式 rank	0
layer_id	模型层编号	12
reuse_distance	重用距离	backward_layer_gap=8
suggested_io	建议 I/O 形态	fixed_shard
version	数据版本	step_10000
checksum	完整性校验	sha256:...

3.5.3 接口实现与协同机制

训练语义接口采用分阶段实现方式。

第一阶段通过目录组织规则、Sidecar Metadata 文件和 I/O Trace 等方式输出训练语义信息，实现训练侧数据分类结果的离线验证。不同类型的数据分别存放于独立目录，并为每次写入生成对应元数据记录。

第二阶段通过用户态库或训练框架插件，在实际 I/O 请求发起时附带语义标签，使训练语义能够与读写请求同步传递。

第三阶段根据 AI SSD 的实际能力，将训练语义进一步映射至驱动层或设备接口，实现训练系统与 AI SSD 的深度协同。

在协同过程中，训练侧负责提供数据类型、生命周期、访问模式和优先级等语义信息；存储侧则根据这些语义信息开展数据放置、缓存管理、调度优化和寿命管理等工作。双方通过统一的元数据格式和 Trace 分析机制形成闭环反馈，不断优化训练过程中的存储访问行为。

通过该接口机制，训练框架内部可见的数据语义能够被存储系统有效利用，为 AI SSD 提供更符合实际训练场景的优化依据，实现训练算法与存储系统的协同优化。

3.6 训练框架适配与原型系统实现



生命周期 Profiler 负责采集训练过程中各类数据对象的创建、访问、释放和写回行为，形成训练数据生命周期画像。采集信息包括对象大小、设备位置、创建时间、释放时间、重用距离以及读写次数等内容，并以结构化日志形式输出。

语义分类模块根据生命周期特征对训练数据进行统一建模，将 Parameter、Gradient、Optimizer State、Activation、Checkpoint、Dataset Cache 和 Log/Profile 等对象映射为标准语义标签。分类结果不仅用于统计分析，同时作为后续调度决策的重要依据。

SSD 友好调度模块根据数据语义和系统资源状态，动态决定数据的 Offload、Prefetch、Write-back 和缓存策略。对于 Activation 采用“重算优先、选择性 Offload”的管理机制；对于 Optimizer State 采用 Fixed Shard 和批量写回策略；对于 Checkpoint 采用异步顺序写和版本管理机制；对于日志和 Profile 数据则采用缓冲与采样策略。

语义接口模块负责将训练侧生成的数据类型、生命周期、访问模式和优先级等信息输出为标准化元数据，为 AI SSD 提供可利用的训练语义输入，实现训练系统与存储系统之间的协同优化。

3.6.3 实验评估与验证机制

为了验证方案有效性，原型系统建立完整的实验评估与验证机制，对训练性能、显存利用率、SSD 友好性以及接口覆盖率等指标进行综合分析。

训练性能指标主要包括 Tokens/s、Step Time、GPU Utilization 和尾延迟等，用于评价调度策略对训练效率的影响；显存相关指标包括 GPU Memory Peak 和 OOM 避免能力，用于评估 Offload 策略的容量扩展效果；存储相关指标包括 Host Writes、平均请求大小、小随机写比例、Offload Wait Time 和 Checkpoint Stall，用于衡量训练 I/O 是否更加符合 SSD 特性。

在验证过程中，系统设置三类对照组，包括 PyTorch 原生训练策略、DeepSpeed 默认 NVMe Offload 策略以及本方案提出的 SSD 友好训练策略。所有实验统一记录模型配置、训练参数、硬件环境和软件版本，确保结果可复现和可比较。

除性能测试外，系统还对 Checkpoint 恢复正确性、语义标签完整性以及 Trace 可解析性进行验证，确保优化策略在提升训练效率的同时保持训练正确性和系统可靠性。最终形成训练性能报告、生命周期分析报告、I/O 行为报告和语义接口评估报告，为后续 AI SSD 联合优化提供依据。

4 技术创新点

4.1 面向 AI SSD 的训练数据语义分类方法

第一项创新是建立面向 AI SSD 的训练数据语义分类方法。与现有训练框架只在内部区分 Tensor 和状态不同，本方案将数据类型、生命周期、访问模式、可重算性、持久化要求和优先级统一为可输出的语义标签，使底层存储侧能够理解训练数据的真实含义。

该方法把训练框架内部可见信息转化为标准化字段，使 AI SSD 能够区分 short_lived activation、long_lived optimizer state、persistent checkpoint 和 read_mostly dataset cache。分类结果不仅用于统计，还直接驱动调度策略。通过这些输出，训练数据不再只是普通 I/O 请求，而是具有可调度、可解释和可验证的语义对象。

4.2 SSD 友好的训练数据 Offload 与调度方法

第二项创新是将 Offload 从容量扩展机制提升为 SSD 友好调度机制。方案在显存、计算和 I/O 之间进行选择，形成参数预取、优化器状态 fixed shard、activation ring buffer、checkpoint 异步顺序写和日志缓冲采样等组合策略。

现有 Offload 技术通常首先解决显存不足问题，关注训练能否运行和显存峰值能否降低。本方案进一步关注 Offload 后的数据是否适合 SSD：写入是否过碎、生命周期是否可区分、是否与 checkpoint 抢占带宽、是否造成关键路径等待。该思路把 Offload 从单一容量手段扩展为显存、计算、I/O 和设备写入压力之间的综合调度问题。

调度方法具备动态降级能力。当 SSD 负载高或 checkpoint 正在写入时，低优先级数据延迟写入；当 GPU 显存压力接近阈值时，选择性 offload 提高优先级；当某类 activation 可重算时，优先使用计算替代保存。该策略能够避免简单 offload 带来的无效写入和训练抖动。

4.3 面向训练过程的 I/O 大块化、顺序化与削减机制

第三项创新是针对训练过程不同数据流设计具体 I/O 形态。通过大块化写入减少小请求，通过顺序化写入降低随机性，通过写入削减减少无效写入，通过优先级控制降低关键路径阻塞。这些机制把训练 I/O 从碎片化模式转化为更适合 SSD 的规整化模式。

大块化机制按数据语义组织，而不是简单合并所有写入。Optimizer state 适合按 rank、layer 或 shard 组织固定块；activation 适合按预分配 ring buffer slot 复用；checkpoint 适合按 rank 分片形成顺序大文件；日志适合按时间窗口缓冲追加。不同组织方式对应不同生命周期和恢复要求，因此需要与语义分类共

同设计。

写入削减机制强调“少写比盲目 offload 更重要”。可重算 activation 不优先写入 SSD，低价值日志不与关键训练 I/O 争用资源，重复更新状态通过缓存和批量写回减少覆盖写，checkpoint 通过保存频率和版本管理控制写入规模。削减策略均以训练正确性和恢复可靠性为前提。

4.4 训练框架与 AI SSD 之间的语义协同接口

第四项创新是定义训练框架到 AI SSD 的语义协同接口。接口内容包括数据类型、生命周期、访问模式、优先级、可重算性、持久化要求和建议 I/O 形态。该接口不依赖具体 SSD 内部实现，但能够为 AI SSD 底层优化提供训练语义来源。

接口采用分阶段设计。第一阶段以 sidecar metadata、目录约定和 trace 文件实现，确保没有专用硬件接口时也能验证语义完整性；第二阶段通过训练框架插件或用户态库在 I/O 发起时携带标签；第三阶段再结合 AI SSD 实际能力映射到底层接口。该路径既保证早期可落地，也为后续硬件协同保留空间。

接口字段区分必选字段和扩展字段。必选字段保证联合测试的最小可用性，扩展字段用于增强策略判断。字段设计避免绑定单一模型和单一设备，使接口可用于不同 Transformer 变体、不同并行策略和不同 SSD 实现。

4.5 基于 PyTorch/DeepSpeed 的可验证原型系统

第五项创新是构建可验证原型系统，而不是停留在概念设计。原型系统基于 PyTorch 和 DeepSpeed，能够运行典型 Transformer 模型，生成生命周期报告和 I/O 报告，并对比默认训练策略和 SSD 友好策略的差异。

原型系统包括生命周期 Profiler、数据分类与标注模块、SSD 友好调度模块、语义接口输出模块和实验评估模块。各模块既可组合运行，也可独立输出报告，便于分阶段验证。原型系统支持策略开关和 baseline 对比，使同一训练任务能够在原生策略、DeepSpeed 默认策略和本方案策略之间切换。

该系统的价值在于可展示、可测试、可迭代。通过 trace、元数据、指标报告和恢复测试，方案能够证明策略是否减少小写和无效写入，是否降低关键路径等待，是否保持训练正确性。

5 实施计划

5.1 第一阶段：训练数据生命周期分析与需求建模

第一阶段完成实验环境搭建、模型选择、插桩工具开发和生命周期分析。验证对象可选取 GPT、BERT、T5 或 LLaMA-like 小中型模型，基于 PyTorch 和 DeepSpeed 采集参数、梯度、优化器状态、activation 和 checkpoint 的生命周期信息。阶段输出包括 Profiler 原型、数据分类模型初稿。

本阶段重点是建立可信观测基础。先在可控规模模型上验证 hook、profiler 和 I/O trace 是否能够稳定采集数据，再逐步引入 DeepSpeed ZeRO 和 NVMe Offload 配置。采集内容包括对象大小、产生阶段、释放阶段、重用距离、读写次数、是否进入 checkpoint、是否可重算以及是否参与训练关键路径。第一阶段重点不在性能提升，而在观测准确性、分类覆盖率和报告可复现性。

5.2 第二阶段：SSD 友好训练调度方法研发

第二阶段完成训练侧调度策略设计与实现。重点研发参数预取策略、优化器状态 fixed shard 策略、activation ring buffer 策略、checkpoint 异步顺序写策略和日志缓冲策略。阶段输出包括调度模块、策略配置文件、DeepSpeed 适配扩展和初步性能对比结果。

本阶段以第一阶段的数据画像为依据，优先处理对 SSD 压力和训练稳定性影响最大的数据流。Optimizer state 重点减少小块和频繁覆盖写；activation 重点建立“重算、保留、SSD offload”三选一策略；checkpoint 重点降低突发写入对训练主循环的阻塞；日志和 profile 重点控制高频小写。

调度模块先采用规则驱动方式，确保策略可解释、可开关、可回归测试。典型规则包括：可重算 activation 优先重算；大尺寸且重用距离较长的 activation 才进入 ring buffer；optimizer state 采用 fixed shard 或批量写回；checkpoint 采用分片异步顺序写；log/profile 采用缓冲采样。随后在 profiler 数据基础上加入代价模型，动态调整阈值和优先级。

5.3 第三阶段：AI SSD 语义接口与协同机制研发

第三阶段完成语义接口定义和联合调试机制建设。训练侧输出数据类型、生命周期、访问模式和优先级标签，并与 AI SSD 侧能力对接接口格式、字段含义和测试流程。阶段输出包括接口规范、元数据样例、联合测试脚本和语义标签覆盖率报告。

本阶段将训练侧分类和调度结果转化为可使用接口。接口规范明确字段名称、字段类型、取值范围、生成时机和对应数据流。元数据样例覆盖 optimizer state、activation、checkpoint、dataset cache 和 log/profile 等典型对象，避免接口只对单一数据类型有效。

验证采用分层方式。首先通过离线 trace 验证语义字段是否能被解析；其次通过训练运行时日志验证标签生成是否稳定；最后在具备条件时通过用户态库、驱动或设备接口进行联调。该路径能够保证接口在不同硬件条件下都有可验证成果。

5.4 第四阶段：系统集成与实验验证

第四阶段完成系统集成和综合验证。系统集成将 Profiler、分类器、调度模块、语义接口和评估脚本串联为完整流程。实验流程包括环境初始化、baseline 训练、策略训练、trace 采集、指标统计、checkpoint 恢复验证和结果报告生成。对于每项策略，应说明启用条件、配置参数、适用数据流和可能代价。

实验验证覆盖功能正确性、性能影响和 SSD 友好性三类内容。功能正确性包括训练是否可运行、loss 是否正常下降、checkpoint 是否可恢复；性能影响包括 tokens/s、step time 和显存峰值；SSD 友好性包括 Host writes、请求大小分布、小随机写比例、checkpoint stall 和 offload wait time。对于无法在当前环境完成的硬件侧指标，应明确说明原因，并以 trace 或软件统计作为替代输入。

表 13 给出阶段计划与成果。

阶段	主要任务	关键成果	验证重点
第一阶段	生命周期采集与建模	Profiler、分类模型初稿	能否识别主要训练数据
第二阶段	SSD 友好调度研发	Offload 调度模块	是否减少小写和无效写
第三阶段	语义接口研发	接口规范、测试样例	标签是否可解析、可统计、可对接
第四阶段	系统集成验证	原型系统、实验报告	训练性能、正确性和 SSD 友好性